

Classificazione di pellicce animali con reti neurali di percettroni multistrato e reti convoluzionali: confronto dei metodi

Davide Rizzi, Claudio de Cesare

Maggio 2026

Indice

Indice	1
1 Cenni teorici	2
Reti Neurali Feed-Forward (FNN)	2
Funzione costo	3
Tensori	3
Reti Neurali Convoluzionali (CNN)	4
2 Dataset	5
Pre-processamento	5
3 Addestramento e validazione	6
Classificazione	6
Regressione	7
4 Risultati	7
5 Conclusioni	8
Riferimenti bibliografici	10

Sinossi

L'obiettivo di questo studio è confrontare le prestazioni di due tipi di rete neurale: le reti neurali dirette (o *feed-forward*) e le reti neurali convoluzionali attraverso la classificazione di immagini di pellicce di dieci diverse specie animali. Per compiti di classificazione multiclasse le reti neurali convoluzionali sono più adatte, raggiungendo un'accuratezza di stima significativamente superiore rispetto alle reti multistrato feed-forward, d'altra parte queste ultime conseguono ottimi risultati per compiti di regressione con una maggiore facilità di implementazione.

1 Cenni teorici

Reti Neurali Feed-Forward (FNN)

Le reti neurali *Feed-Forward* (FNN), o *Multi-Layer Perceptron* (MLP), rappresentano l'architettura fondamentale del deep learning. In questa configurazione, le informazioni viaggiano in una sola direzione: dai nodi di ingresso, attraverso gli strati nascosti (*hidden layers*), fino ai nodi di uscita.

La rete è formata da $L \in \mathbb{N}$ strati in cui con 0 viene indicato quello di input. Sia n_l il numero di neuroni allo strato $l \in \{0, \dots, L\}$:

- **Input:** L'input della rete è un vettore colonna $x = a_0 \in \mathbb{R}^{n_0}$;
- **Pesi:** Ogni strato $l \in \{1, \dots, L\}$ possiede una matrice contenente i pesi della rete $W_l \in \mathbb{R}^{n_l \times n_{l-1}}$. L'elemento w_{ij}^l rappresenta il peso della connessione dal neurone j dello strato $l-1$ al neurone i dello strato l ;
- **Distorsione (Bias):** Ogni strato $l \in \{1, \dots, L\}$ possiede un vettore colonna del bias $b_l \in \mathbb{R}^{n_l}$.

Propagazione in avanti

La propagazione in avanti calcola l'output della rete applicando ricorsivamente trasformazioni lineari seguite da trasformazioni non lineari. Per ogni strato $l \in \{1, \dots, L\}$ si eseguono i seguenti passaggi:

1. **Trasformazione lineare (Pre-Attivazione):** Si calcola il vettore $z_l = W_l a_{l-1} + b_l \in \mathbb{R}^{n_l}$;
2. **Trasformazione non lineare (Attivazione):** Si applica una funzione in genere non lineare σ_l che opera elemento per elemento per ottenere $a_l = \sigma_l(z_l) \in \mathbb{R}^{n_l}$. L'importanza della non linearità di σ_l risiede nel fatto che la rete altrimenti collasserebbe in un'unica grande funzione lineare.

Le funzioni di attivazione più comuni e implementate sono:

- **Sigmoide:** Mappa i valori nell'intervallo $(0, 1)$ e soffre del problema della scomparsa del gradiente.

$$\varsigma(z) = \frac{1}{1 + e^{-z}} \text{ con } \varsigma'(z) = \varsigma(z)(1 - \varsigma(z))$$

- **Tangente iperbolica:** Mappa i valori nell'intervallo $(-1, 1)$, centrando i dati rispetto a zero.

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \text{ con } \tanh'(z) = 1 - \tanh^2(z)$$

- **Rettificatore (ReLU):** Risolve il problema della scomparsa del gradiente, non è tuttavia limitata.

$$\text{ReLU}(z) = \max(0, z)$$

Verrà indicata con σ_l la funzione di attivazione scelta allo strato l . Per lo strato di output si possono usare anche le seguenti funzioni di attivazione:

- **Esponenziale normalizzata (Softmax):** L'output rappresenta una distribuzione di probabilità. Per l' i -esima componente dell'output:

$$\sigma_i = \sigma(z)_i = \frac{e^{z_i}}{\sum_{k=1}^{n_L} e^{z_k}} \text{ con } \frac{\partial \sigma_i}{\partial z_j} = \sigma_i(\delta_{ij} - \sigma_j)$$

- **Lineare:** $\sigma(z) = z$.

Retropropagazione

L'addestramento avviene aggiornando i parametri W_l e $b_l \forall l \in \{1, \dots, L\}$ per minimizzare la funzione costo $\mathcal{L}$. La regola di aggiornamento varia a seconda dell'algoritmo di ottimizzazione usato; per gli algoritmi basati sul gradiente sono necessarie $\frac{\partial \mathcal{L}}{\partial W_l}$ e $\frac{\partial \mathcal{L}}{\partial b_l}$. Per calcolare i gradienti viene usata la regola della catena (*Chain Rule*) definendo l'errore dello strato l come $\delta_l = \frac{\partial \mathcal{L}}{\partial z_l} \in \mathbb{R}^{n_l}$:

1. **Errore strato L :**

$$\delta_L = \nabla_{a_L} \mathcal{L} \odot (\sigma_L)'(z_L)$$

2. **Errore negli strati $l = 1, \dots, L - 1$:**

$$\delta_l = ((W_{l+1})^\top \delta_{l+1}) \odot (\sigma_l)'(z_l)$$

3. **Gradiente dei parametri:**

$$\frac{\partial \mathcal{L}}{\partial W_l} = \delta_l (a_{l-1})^\top \in \mathbb{R}^{n_l \times n_{l-1}}$$

$$\frac{\partial \mathcal{L}}{\partial b_l} = \delta_l$$

Funzione costo

Date le predizioni della rete a_L e le etichette y , le principali funzioni costo (*loss*) sono:

- **Errore quadratico medio (MSE):** $\mathcal{L} = \frac{1}{2} \|a_L - y\|_2^2$
- **Errore assoluto medio (MAE):** $\mathcal{L} = \frac{1}{n_L} \sum_{k=1}^{n_L} |a_L^k - y^k|$
- **Entropia incrociata (Cross-Entropy):** In questo caso l'output è dato dalla funzione Softmax in modo che a_L derivi da una distribuzione di probabilità e le etichette y siano rappresentate come vettori binari di stati finiti (*one-hot*) in cui tutti gli elementi sono 0 all'infuori dell'elemento nella posizione che individua la classe corretta che è 1. :

$$\mathcal{L} = - \sum_{k=1}^{n_L} y^k \log(a_L^k)$$

Tensori

Un tensore d'ordine N è un elemento del prodotto tensoriale di N spazi vettoriali, cioè $T \in \bigotimes_{j=1}^N V_{I_j}$.

Se le dimensioni degli N spazi vettoriali V_{I_j} sono finite e $\dim V_{I_j} = I_j$ un tensore reale T di ordine N si può identificare come vettore dello spazio vettoriale $\mathbb{R}^{I_1 \times \dots \times I_N}$

$$T = \sum_{i_1=1}^{I_1} \dots \sum_{i_N=1}^{I_N} T_{i_1 \dots i_N} (e_{i_1} \otimes \dots \otimes e_{i_N})$$

In virtù di questo isomorfismo è possibile rappresentare un tensore T come vettore contenente tutti i suoi elementi (analogamente a quanto accade per le matrici) o come matrice contenente la giustapposizione delle sue fette. Questo tipo di rappresentazione può avvenire in N maniere distinte, per cui dato un tensore $T \in \mathbb{R}^{I_1 \times \dots \times I_N}$, esso viene matricizzato modo- m nella matrice $T_{[m]} \in \mathbb{R}^{I_m \times (I_1 \dots \hat{I}_m \dots I_N)}$ indicando con $\hat{I}_m$ la marginalità dell'indice.

Reti Neurali Convoluzionali (CNN)

Dal punto di vista teorico, le Reti Neurali Convoluzionali (CNN) sono progettate per elaborare tensori che presentano una topologia a griglia spaziale (es. immagini 2D). Mentre in una rete Feed-Forward standard ogni neurone di output interagisce con ogni neurone di input, le CNN si basano su due principi matematici fondamentali che riducono drasticamente i parametri: la connettività locale (sparsità) e la condivisione dei parametri.

Strato Convoluzionale

Sia $X \in \mathbb{R}^{H \times W \times C_{in}}$ il tensore di input (altezza, larghezza e numero di canali) e sia $K \in \mathbb{R}^{h \times w \times C_{in}}$ un singolo filtro (o kernel) locale. La mappa delle caratteristiche bidimensionale $Y \in \mathbb{R}^{H' \times W'}$ prodotta da quel filtro è definita come una correlazione incrociata:

$$Y_{i,j} = \sum_{m=0}^{h-1} \sum_{n=0}^{w-1} \sum_{c=0}^{C_{in}} X_{i \cdot s_h + m, j \cdot s_w + n, c} K_{m,n,c} + b$$

dove:

- $s_w, s_h \in \mathbb{N}$ è il passo di traslazione del filtro sulla griglia (*stride*);
- $b \in \mathbb{R}$ è il bias additivo scalare;
- H' e W' le dimensioni spaziali dell'output calcolate analiticamente introducendo un eventuale padding p_h e p_w (zeri aggiunti ai bordi dell'input):

$$H' = \left\lfloor \frac{HWC_{in} - h + 2p_h}{s_h + 1} \right\rfloor, \quad W' = \left\lfloor \frac{HWC_{in} - w + 2p_w}{s_w + 1} \right\rfloor$$

Applicando C_{out} filtri si otterrà un tensore $Y \in \mathbb{R}^{H' \times W' \times C_{out}}$. Come per le FNN, al tensore di output si applica una funzione di attivazione elemento per elemento (tipicamente ReLU) ottenendo

$$A_{i,j,k} = \max(0, Y_{i,j,k})$$

Strato di Pooling (Sottocampionamento)

Il *pooling* sostituisce l'output della rete in una certa posizione con una statistica riassuntiva degli interni limitrofi. Il più utilizzato è il Max Pooling, che introduce una locale invarianza spaziale e riduce il costo computazionale scartando le informazioni posizionali esatte in favore della semplice presenza di una *feature*. Data una regione spaziale quadrata di lato f e uno *stride* s_p , l'operazione estrae il massimo locale:

$$P_{i,j,c} = \max_{0 \leq m,n < f} A_{i \cdot s_p + m, j \cdot s_p + n, c}$$

l'operazione avviene canale per canale in modo indipendente.

Classificatore finale

Dopo una cascata di strati Convoluzionali e di Pooling, il volume di dati viene "appiattito" (operazione di flattening). Il tensore multidimensionale collassa in un singolo vettore colonna $v \in \mathbb{R}^N$ per poter essere passato in input a uno o più strati lineari completi (Fully Connected), calcolando infine l'output tramite Softmax per la classificazione statistica.

Retropropagazione

Le differenze matematiche e strutturali fondamentali nella retropropagazione tra CNN e FNN si riassumono in tre dinamiche:

- **Calcolo del Gradiente (Accumulo vs Singolo Nodo):** Nelle FNN, il gradiente di un peso deriva da una singola connessione calcolata tramite prodotto esterno ($\frac{\partial \mathcal{L}}{\partial W} = \delta a^\top$). Nelle CNN, poiché lo stesso kernel scorre sull'intero input (*weight sharing*), il gradiente di un singolo peso è la somma degli errori su tutte le posizioni spaziali in cui ha operato, calcolata come una correlazione incrociata:

$$\frac{\partial \mathcal{L}}{\partial K_{m,n}} = \sum_i \sum_j \delta_{i,j}^Y X_{i+m,j+n} \text{ con } \delta_{i,j}^Y = \frac{\partial \mathcal{L}}{\partial Y_{i,j}}$$

- **Propagazione dell'Errore (Convoluzione vs Matrice Trasposta):** Nelle FNN, l'errore viene retropropagato allo strato precedente moltiplicandosi banalmente per la matrice dei pesi ($W^\top \delta$). Nelle CNN, la propagazione dell'errore richiede l'esecuzione di una convoluzione completa (*full convolution*) tra la mappa degli errori δ e il kernel ruotato spazialmente di 180 gradi.
- **Gestione del Pooling:** Le FNN hanno solo strati addestrabili. Le CNN utilizzano livelli senza parametri come il Max Pooling. Durante la retropropagazione, questo strato agisce da semplice "selettore": instrada l'intero gradiente ricevuto in ingresso esclusivamente alla coordinata spaziale che conteneva il valore massimo durante il forward pass, assegnando un gradiente pari a 0 a tutte le altre posizioni della finestra.

2 Dataset

I dati analizzati nella classificazione sono stati ottenuti da diverse fonti naturalistiche accreditate e aggregati nel dataset SPOTS10. Essi comprendono 50 000 immagini uniche e non completamente sovrapposte di dimensione 32×32 px in scala di grigi delle macchie tipiche delle pellicce di 10 diverse specie di animali: ghepardo, cervo, giraffa, iena, giaguaro, leopardo, tapiro, tigre, squalo balena e zebra. Lo scopo dello studio è addestrare un modello che riconosca correttamente a quale animale appartiene il manto riportato in ciascuna fotografia. Ogni classe contiene 5000 immagini, l'80% delle quali costituisce l'insieme di addestramento del modello, la restante parte l'insieme di valutazione.

Per mostrare le prestazioni del perceptrone multistrato su un caso d'uso diverso, vale a dire una regressione, è stata fissata la funzione da approssimare $f(x) = \log(20x^2 + 5) - 11 \sin^2(\frac{3}{2}\sqrt{x})$ in maniera tale che fosse altamente non lineare, sono stati simulati 1500 campioni numerici di addestramento e 300 di valutazione con ascissa estratta da una distribuzione uniforme sull'intervallo di riferimento $[0, 35]$ e ordinata corrispondente perturbata da rumore bianco gaussiano $\mathcal{N}(0; 1, 2)$.

Pre-processamento

I dati compressi sono stati estratti tramite una funzione python, essi sono costituiti da coppie di matrici e interi. Le immagini sono rappresentate in forma matriciale con elementi interi che quantificano la luminosità del pixel (nell'intervallo $[0, 255]$) individuato dalla posizione dell'elemento nella matrice, le etichette invece sono variabili categoriche intere (nell'intervallo $[0, 9]$). Per agevolare l'analisi, i dati matriciali sono stati salvati in precisione singola (*float32*) e normalizzati nell'intervallo $[0, 1]$.

Nella valutazione del modello convoluzionale le immagini sono state trasformate in tensori di ordine 3, le etichette in tensori di ordine 0, tutte le informazioni topologiche sulle configurazioni sono state così preservate.

D'altra parte per la rete neurale di perceptroni multistrato le immagini sono state trasformate in vettori colonna (in convenzione Fortran), con la conseguente perdita di informazioni sul naturale schema di disposizione delle macchie, le etichette sono state invece trasformate in vettori *one-hot*.

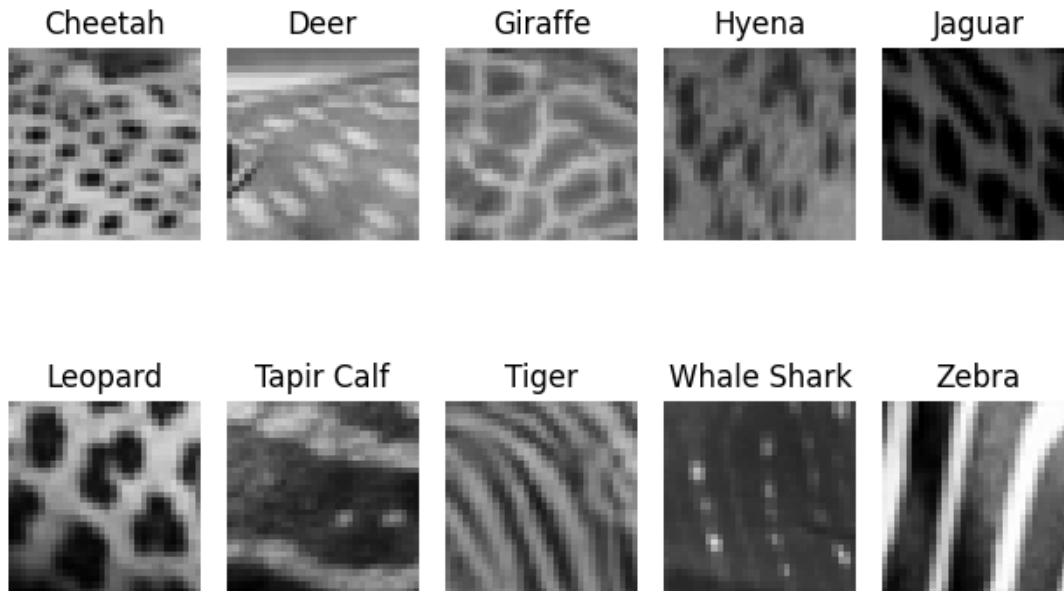


Figura 1: Esempi di immagini

La pre-elaborazione dei dati di regressione invece ha comportato la standardizzazione delle ordinate per prevenire un eventuale sbilanciamento dei pesi causato dalle forti oscillazioni della funzione del modello campionario.

3 Addestramento e validazione

Affinché sia garantita la riproducibilità dello studio, è stato impostato 42 come seme iniziale per la generazione dei numeri pseudocasuali.

Classificazione

- **Feed-Forward:** La FNN possiede due strati nascosti, rispettivamente con 2^8 e 2^7 neuroni, per ottimizzare l'elaborazione di macchina, per un totale di 4 strati (compresi quello di *input*, formato da 1024 neuroni rappresentanti i pixel, e quello di *output*, formato da 10 neuroni rappresentanti le classi). La funzione di attivazione interna scelta è la *ReLU* per efficienza computazionale, mentre per l'output si è optato per la *softmax*, in quanto più adatta a rappresentare la probabilità di appartenenza a ciascuna classe.
- **Convolutionale:** La CNN è costituita da 3 blocchi convoluzionali, ciascuno dei quali è formato da 32 kernel di dimensione 3×3 . Poiché il dataset è costituito da tensori bidimensionali quadrati, si è reso necessario per tutti i blocchi un unico parametro di *stride* e *padding*, pari rispettivamente a 2 e 1. Nello specifico, ciascun blocco convoluzionale effettua un sottocampionamento di tipo *MaxPooling*. Come funzione di attivazione per lo strato convoluzionale e per lo strato del classificatore è stata scelta la *ReLU*, mentre per l'output finale è stata scelta la *softmax*, entrambe per gli stessi motivi citati per la FNN.

Come funzione di costo è stata scelta la *cross-entropy*, a causa delle sue proprietà di discriminazione informativa. Il tasso di apprendimento adoperato è stato $\eta = 10^{-3}$; la cardinalità delle partizioni dell'insieme di addestramento nel metodo dei *mini-batch* è stata posta pari a 2^5 e il numero di epoche è stato limitato a 21.

Regressione

Sebbene nella classificazione una rete di perceptron multistrato non riesca a riconoscere accuratamente le macchie degli animali, mantenendo fissi il numero di strati e di neuroni una rete dello stesso tipo raggiunge ottime prestazioni nello svolgimento di compiti di regressione. Si è utilizzata la medesima architettura, formata da un neurone in *input* e uno in *output*, e due strati interni rispettivamente di 2^8 e 2^7 neuroni. Per lo strato d'uscita è stata scelta la funzione di attivazione lineare e per la funzione di costo l'errore quadratico medio, lasciando invariata la dimensione dei lotti a 2^5 . I restanti iperparametri, vale a dire il tasso di apprendimento e la funzione di attivazione per gli strati nascosti, sono stati scelti tramite convalida incrociata con 10 partizioni dell'insieme di addestramento (*K-fold Cross Validation*). Dopo 100 epoche sono stati selezionati i parametri $\eta^* = 10^{-3}$ e $\sigma^*(x) = \tanh(x)$, associati al minor errore quadratico medio.

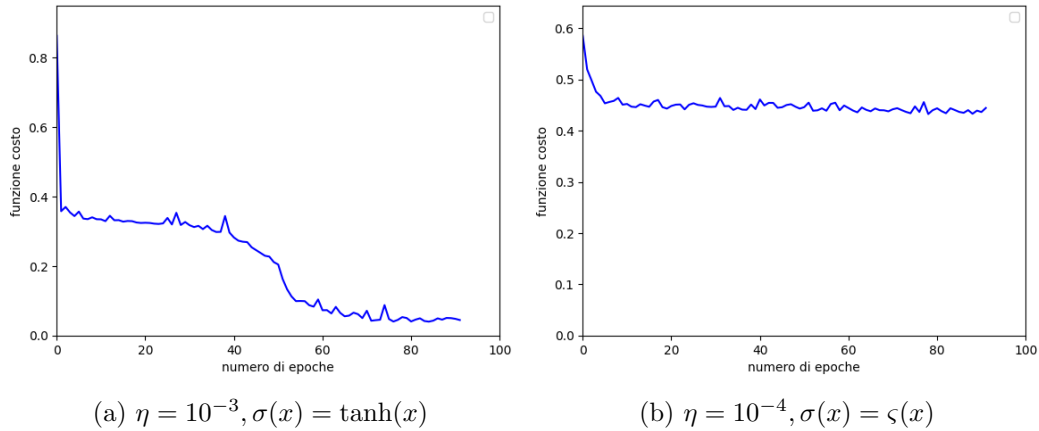


Figura 2: Grafici della funzione costo in fase di cross validation

4 Risultati

La rete neurale convoluzionale è giunta a convergenza dopo 6 minuti con un'accuratezza di 80,61%, invece la rete neurale *feed-forward* utilizzando l'algoritmo dei momenti adattivi ha impiegato 50 minuti per convergere fermandosi a un'accuratezza del 31,30% e utilizzando l'algoritmo del gradiente stocastico 43,5 minuti per la convergenza con un'accuratezza di appena 27,00%.

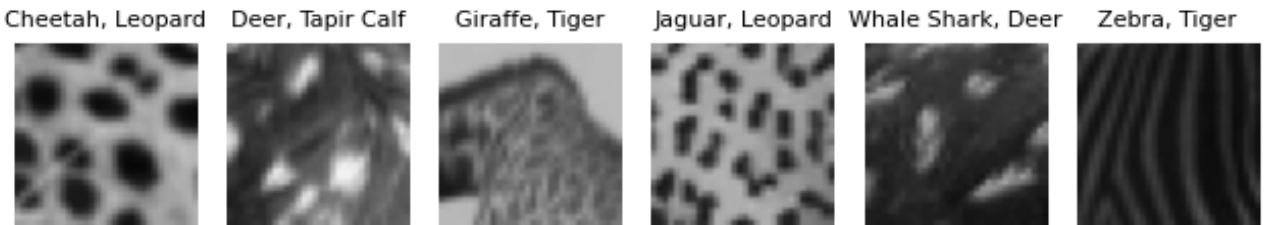


Figura 3: Errori di classificazione CNN (Etichetta, Previsione)

È possibile escludere il rischio di sovradattamento (*overfitting*) del modello poiché dopo avergli fornito in *input* un sottoinsieme delle immagini utilizzate per l'addestramento è stato notato che l'accuratezza e la *loss* non miglioravano in maniera significativa. Per questa ragione si può supporre che la FNN richieda più risorse (condizione che si concretizza in un maggior numero di neuroni, strati o epoche) rispetto a una CNN per lo svolgimento di uno stesso compito affinché l'accuratezza di classificazione sia

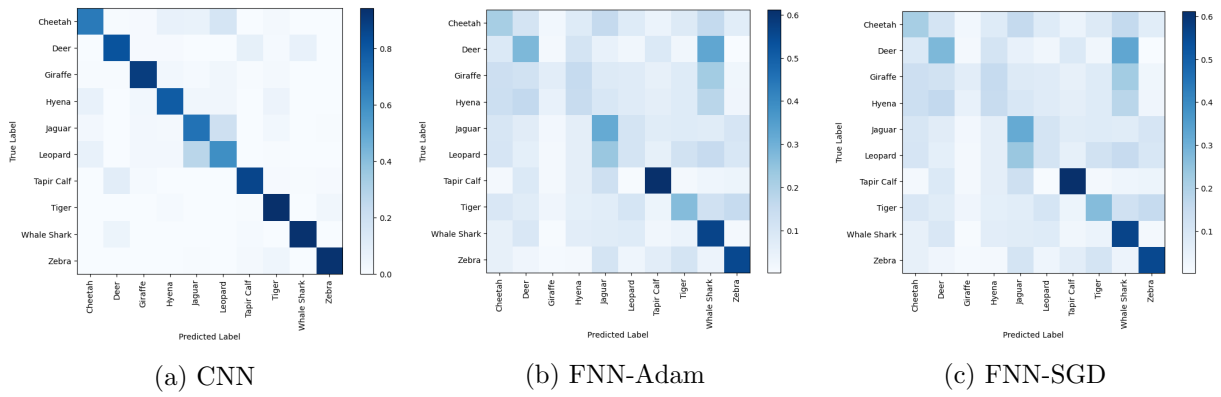


Figura 4: Confronto tra matrici di confusione delle reti, con etichetta in ordinata e previsione in ascissa

identica.

D'altra parte la medesima rete *feed-forward*, impiegata per la regressione con gli iperparametri specificati in precedenza, si è adattata ai campioni di verifica in 30 secondi dopo 150 epoche (in realtà si evince che già dopo 75 epoche la *loss* è stazionaria) con errore quadratico medio (corrispondente alla devianza residua) $MSE = RSS = 1,555$ e coefficiente di determinazione $R^2 = 0,915$ raggiungendo dunque ottimi risultati.

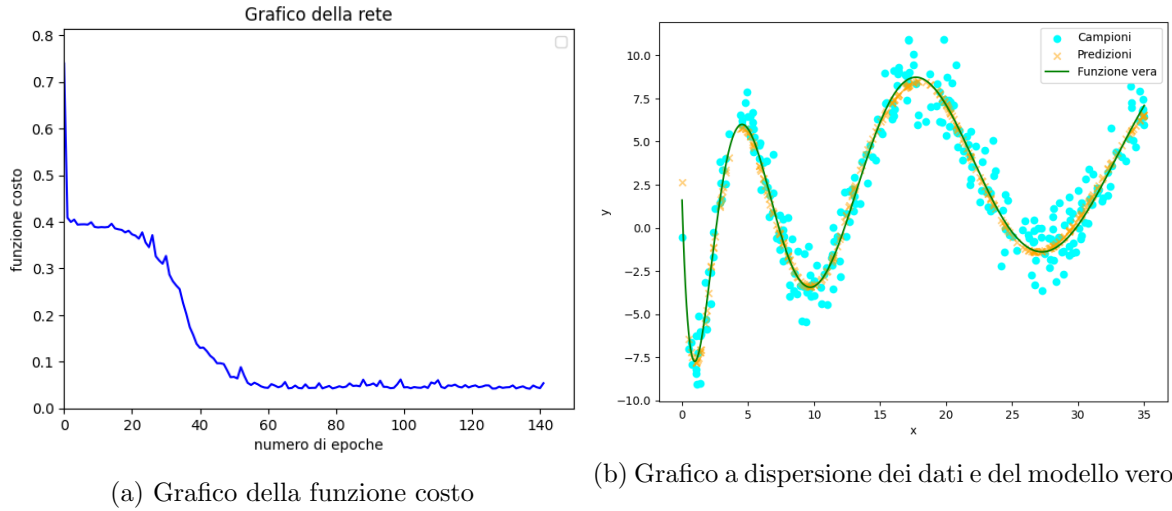


Figura 5: Grafici di regressione

5 Conclusioni

L'analisi ha mostrato che, per compiti di classificazione multiclasse e a parità di iperparametri, le reti neurali convoluzionali sono caratterizzate da prestazioni significativamente superiori (accuratezza di 80,61% raggiunta in 6 minuti) rispetto alle reti neurali dirette (accuratezza di 31,30% raggiunta dopo 50 minuti). Ciononostante, queste riescono agevolmente a risolvere problemi di regressione anche fortemente non lineare producendo ottimi risultati dal punto di vista dell'analisi statistica, come il coefficiente di determinazione $R^2 = 0,915$ che suggerisce un'aderenza superiore al 90% del modello al fenomeno descritto dai dati.

Appendice

I codici dell'analisi sono stati redatti mediante python, sfruttando le librerie seguenti: *numpy*, *matplotlib*, *torch*, *os*, *array*, *struct*, *scikit-learn*, *random*, *itertools*. Le reti neurali dello studio sono state inoltre appositamente definite tramite classi, adoperando come metodi gli algoritmi del gradiente stocastico con *mini-batch* e dei momenti adattivi.

Algorithm 1: Gradiente stocastico (SGD) con *mini-batch*

Input: Insieme di addestramento $\mathcal{D} = \{(x_i, y_i)\}_{i=1, \dots, N}$, funzione obiettivo $\mathcal{L}(\theta; x, y)$, tasso di apprendimento η , numero di epoche T , cardinalità di partizione $M < N$
Output: Parametri ottimizzati θ^*

```
1 Inizializzazione  $\theta_0 = 0$ ;  
2  $t = 0$ ;  
3 for  $t < T$  do  
4   Incrementa il contatore dei passi:  $t \leftarrow t + 1$ ;  
5   Mescola casualmente l'insieme di addestramento  $\mathcal{D}$ ;  
6   Partiziona  $\mathcal{D}$  in  $k = \frac{N}{M}$  sottoinsiemi disgiunti (mini-batch)  $\mathcal{B}_1, \dots, \mathcal{B}_k$  t.c.  $|\mathcal{B}_j| = M \quad \forall j = 1, \dots, k$ ;  
7   foreach  $\mathcal{B}_j$  do  
8     Calcola il gradiente stimato sul lotto:  $g_j^t \leftarrow \frac{1}{M} \sum_{i=1}^M \nabla_{\theta} \mathcal{L}(\theta; x_i, y_i)$ ;  
9     Aggiorna stima del punto di minimo  $\theta_t \leftarrow \theta_t - \eta g_j^t$ ;  
10  end  
11 end  
12 return  $\theta_t$ ;
```

Algorithm 2: Momenti adattivi (Adam) con *mini-batch*

Input: Insieme di addestramento $\mathcal{D} = \{(x_i, y_i)\}_{i=1, \dots, N}$, funzione obiettivo $\mathcal{L}(\theta; x, y)$, tasso di apprendimento η , decadimenti esponenziali $\beta_1, \beta_2 \in [0, 1)$, cardinalità partizione $M < N$, costante numerica ϵ , numero di epoche T e parametri iniziali θ_0 .
Output: Parametri ottimizzati θ^*

```
1 Inizializzazione di primo e secondo momento  $m_0 \leftarrow 0, v_0 \leftarrow 0$ ;  
2  $t \leftarrow 0$ ;  
3 for  $t < T$  do  
4   Incrementa il contatore dei passi:  $t \leftarrow t + 1$ ;  
5   Mescola casualmente l'insieme di addestramento  $\mathcal{D}$ ;  
6   Partiziona  $\mathcal{D}$  in  $k = \frac{N}{M}$  sottoinsiemi disgiunti (minibatch)  $\mathcal{B}_1, \dots, \mathcal{B}_k$  t.c.  $|\mathcal{B}_j| = M \quad \forall j = 1, \dots, k$ ;  
7   foreach  $\mathcal{B}_j$  do  
8     Calcola il gradiente stimato sul lotto:  $g_t \leftarrow \frac{1}{M} \sum_{i=1}^M \nabla_{\theta} \mathcal{L}(\theta; x_i, y_i)$ ;  
9     Aggiorna stima del primo momento:  $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ ;  
10    Aggiorna stima del secondo momento non centrato:  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ ;  
11    Correggi il bias del primo momento:  $\hat{m}_t \leftarrow \frac{m_t}{1 - \beta_1^t}$ ;  
12    Correggi il bias del secondo momento:  $\hat{v}_t \leftarrow \frac{v_t}{1 - \beta_2^t}$ ;  
13    Aggiorna i parametri:  $\theta \leftarrow \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$ ;  
14  end  
15 end  
16 return  $\theta_t$ ;
```

Riferimenti bibliografici

- [1] John Atanbori. *SPOTS-10: Animal Pattern Benchmark Dataset for Machine Learning Algorithms*. ArXiv: 2410.21044, 2024.
- [2] Yoshua Bengio, Ian Goodfellow, Aaron Courville, et al. *Deep learning*, volume 1. MIT press Cambridge, MA, USA, 2017.
- [3] Michael A Nielsen. *Neural networks and deep learning*, volume 25. Determination press San Francisco, CA, USA, 2015.
- [4] Keiron O'Shea and Ryan Nash. *An Introduction to Convolutional Neural Networks*. ArXiv: 1511.08458, 2015.